

12 | 页面实战：如何搭建一个电商首页？

2022-4-22 蒋宏伟



你好，我是蒋宏伟。

今天这一讲就是基础篇中的最后一讲了，还记得我在基础篇的第一讲中和你说的吗？刚刚开始学习的时候，不要一头扎进技术的细节中去学习，应该拿起 React Native 的知识地图先看看，知道自己学习的方向并给自己树立一个学习目标。

现在，我们的基础篇一个月的学习已经接近尾声了，是时候给自己做一个阶段性的总结了！回头看看，自己当初的学习目标有没有达成，又有哪些知识掌握得好，还有哪些知识还需要补足。

俗话说，实践是检验真理的唯一标准。还记得我们在[01讲](#)中制定的学习目标吗？当初我们的学习计划，就是能够使用 React Native 搭建一个简易的电商首页。因此，基础篇的大作业就是“搭建一个简易的电商首页”，希望你能够认真完成这次大作业，好好检查一下自己都学到了什么。

这一讲，我不会讲具体的代码实现，主要讲的是我在“搭建一个简易的电商首页”时的技术设计思路，希望我的思路能够对你的实现基础篇的大作业有所帮助。

建议你在学完这一讲后，先参考这一讲的思路自己实现一遍，然后再去看 [GitHub](#) 上的参考答案。

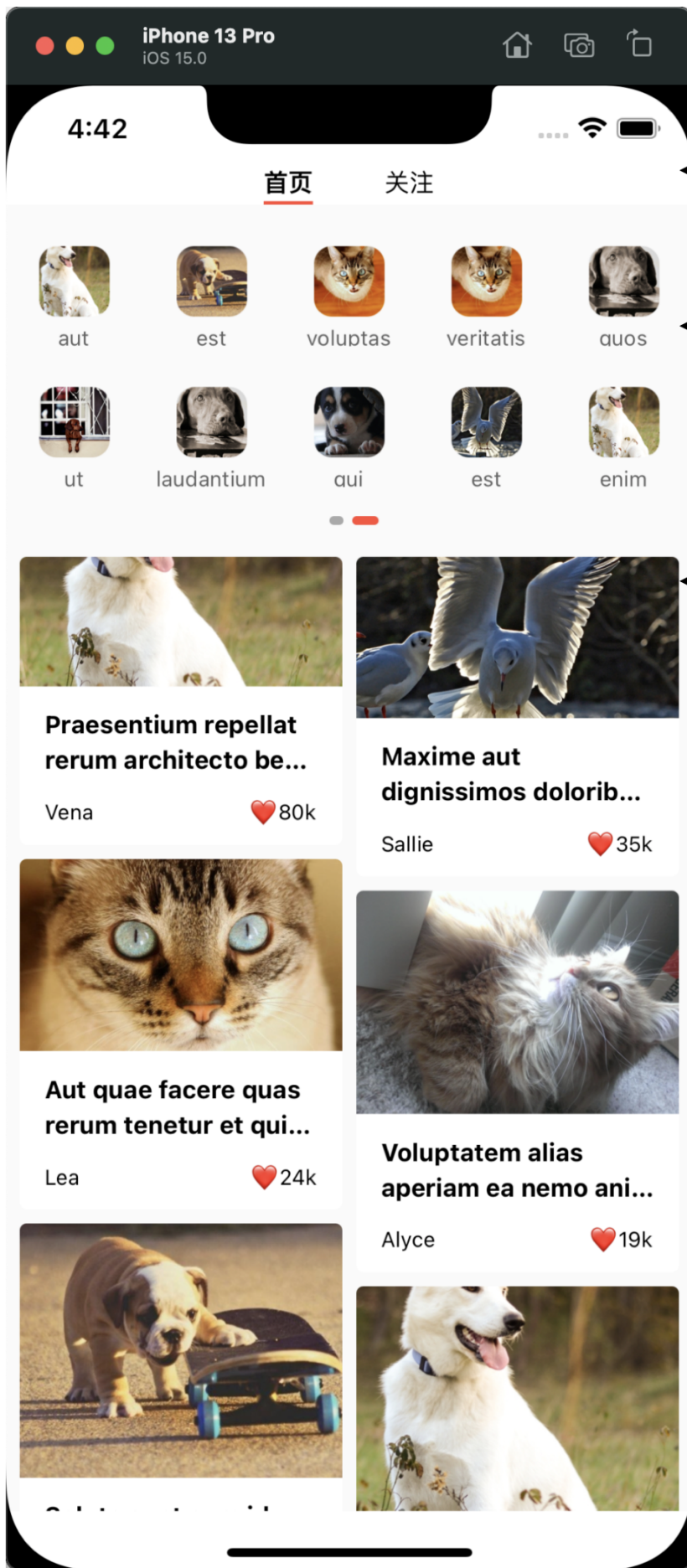
简易电商首页

为了让开发这个简易电商首页显得更有意思一些，我先从产品角度给你讲讲它的背景。

故事是这样的，你的老板最近赶上了 NFT（非同质化代币，Non-Fungible Token）的风口，现在卖 JPG 也能赚大钱了，老板靠着这个点子融来了一笔钱，准备大干一场。你的老板让产品和 UI 同学参考业内的电商 App，做出了一个设计原型，接着就轮到你上场了。

你作为团队中的核心成员，分配到的任务是**搭建 App 的首页**，App 的其他部分由其他同事负责。考虑到要快速上线，你技术领导准备**用 React Native 来实现**，现在是你大展身手的时候了。

团队的设计同学，这时候把首页设计稿交付到你了，设计稿如下：



顶栏

用于切换首页和关注页

金刚位

由 $2 * 10$ 个图标组成
支持左右滑动切换

瀑布流

由高度不定的卡片组成
卡片内容包括：

1. JPG 本身
2. JPG 的座右铭
3. 作者和点赞人数

你可以看到，App 首页的主要功能包括三个部分：顶栏、金刚位、瀑布流。

顶栏是固定在首页中的，它的主要功能是用于切换首页和关注页，其中关注页不是你负责，因此顶栏你只需要注意两点，第一能够支持点击切换，第二顶栏要始终保持在顶部，页面滚动的时候需要保持不动。

金刚位是其他功能页的核心入口，它横跨两个屏幕，每屏幕两行，每行 5 个图标。金刚位的特点是，它自身支持左右滑动切换，并且在页面滚动时金刚位也要跟随着一起滚动。

瀑布流是 JPG 的核心展示区，它由若干个高度不确定的卡片组成，每一批卡片 20 个，卡片数据是从后端请求过来的，并且需要支持无限滚动。

你可以先停下来，思考一下这个项目你会怎么设计，你会怎么写代码。好了，接下来我会把我的设计思路告诉你。

项目结构

我以前和你介绍过，搭建页面讲究的是代码未动构思先行。动手开发，我一般会从三个技术维度进行思考，**项目维度**、**页面维度**和**单个组件维度**，主要围绕技术选型、可行性、可扩展性、可维护性这些方向进行。

遇到大的需求，我还会专门先写技术文档，内容包括核心技术选型、组件拆分方式、组件之间的关系、状态的数据结构和流程图，等等。写文档的过程也就是把模糊的构思变成清晰的文字的过程，在这个过程中，我会找出一些以前没有思考到的要点，把风险提前暴露出来，同时写文档也能帮我把设计思路变得更有条理一些。

回到 NFT 电商项目，在项目维度我们先围绕技术选型、可行性、可扩展性、可维护性这些方面思考一下。

首先，在开发语言的选择上，后续这肯定是大型项目，直接选 TypeScript。说实话，在用习惯 TypeScript 之后，要我换回 JavaScript，我估计我自己的开发效率会变得更低，代码 BUG 也会更多。TypeScript 静态类型检查真的很好用，推荐给你，即使你现在不会，也请你把 TypeScript 列到你的必学清单中。

第二，在状态管理的技术选型上，页面级别的，我们使用 useState 就够了，只有大型应用中，我才会考虑使用 Redux，现在直接用 Redux 有点重了，后面再引进来也不迟。

接下来就是列表组件的技术选型，现在这个时间点，我还是会用 RecyclerView。当然这有个难点，怎么实现金刚位和瀑布流的混合列表，以及怎么在瀑布流中实现不定高布局呢？这些实现细节我们可以后面再思考。可能有些人觉得这非常难，如果是工作中，就需要专门安排人进行技术攻关了，当然这也是可以的。

我认为在项目维度上，更值得和你探讨的问题是：**项目目录应该如何设计，才能支撑后续项目变大的可扩展性和可维护性？**从单个 NFT 首页本身来讲，我的设计思路是这样的，你可以参考一下：

```
.
├── api
│   └── homeAPI.tsx
├── components
│   ├── Grid
│   └── RecyclerView
├── utils
├── features
│   ├── Icons
│   ├── List
│   ├── TopBar
│   └── WaterFallCard
└── index.tsx
```

整体设计思路是把通用代码放到最外面的 api、components、utils 目录，把纯业务相关的功能代码收拢在 features 目录中。这些目录的具体作用如下：

api：后端约定好的接口地址不容易变，因此我把请求后端接口的函数都放到了 homeAPI.tsx 的文件中了；

components：开发页面中能够沉淀下来，后续可能复用的组件，我会放到 components 文件夹中。比如，金刚位中 2 * 5 的图标，它的布局方式就是网格布局，那就可以抽离一个通用的网格布局组件 Grid；

utils：通用工具函数；

features：业务组件和其后端接口数据的处理逻辑部分，它们是最容易变动的，而且关联性很强，因此我把它们看作一个功能，有时候代码行数不多我也会偷懒不拆，直接把这组件和组件的后端数据处理逻辑放到同一个文件中。按功能 feature 拆分而不是按组件本身进行拆分的思路，我是从 [Redux 的最佳实践](#)中学来的。

“Structure Files as Feature Folders with Single-File Logic”，相同 feature 的文件，都放在同一个文件夹下。

index.tsx：页面的根组件，用 index.tsx 的原因是引用起来更加方便，可以少写一层引用路径。

这时你可能会问，这种项目的结构设计，可扩展性怎样？开发页面用这个项目结构是可以，但咱们不是要开发 NFT 的 App 嘛。

其实，这就是个套娃的过程了，当然里面也有一些技巧，如果你后续要开发一个完整的 App，我的扩展设计思路如下：

```
.
├── api
├── components
├── packages
├── utils
├── features
├── screen
│   ├── Home
│   │   ├── api
│   │   ├── components
│   │   └── ...
│   └── Follow
└── index.tsx
```

在上面的项目结构中，首页 Home 的页面结构也是 api、components、features、utils、index.tsx 的结构，只不过用 Home 目录包裹起来了，并将其放到了 screen 目录中。一个文件具体放哪儿一层，按照通用程度来划分：

页面级别的共享：我会放在 ./screen/Home/api、./screen/Home/components 等目录下；

应用级别的共享：一个应用中有多个页面，多个页面之间的共享我会放在 ./api、./components 等目录下；

项目级别的共享：有时候项目和项目之间的代码也是会共用的，这部分代码我会放在 packages 目录下，并通过 npm 的方式进行分发。这个思路，我参考的是业内的

页面拆分

项目维度弄清楚后，接下来我重点思考的问题是如何“拆稿”，也就是把 UI 设计稿拆成组件，特别是要把组件状态确认好。

我拿到 UI 设计稿后，发现了两个我熟悉的通用组件，这些通用组件是我以前写代码时沉淀下来的一些应用级别的共享代码。虽然这些 UI 组件在每个 App 上都长得不一样，很难做成多项目通用、业内通用的组件，但自己做项目时直接拿过来改改，还是非常好用的。你看，**以前通用组件、通用工具的积累，现在派上用场了吧。**

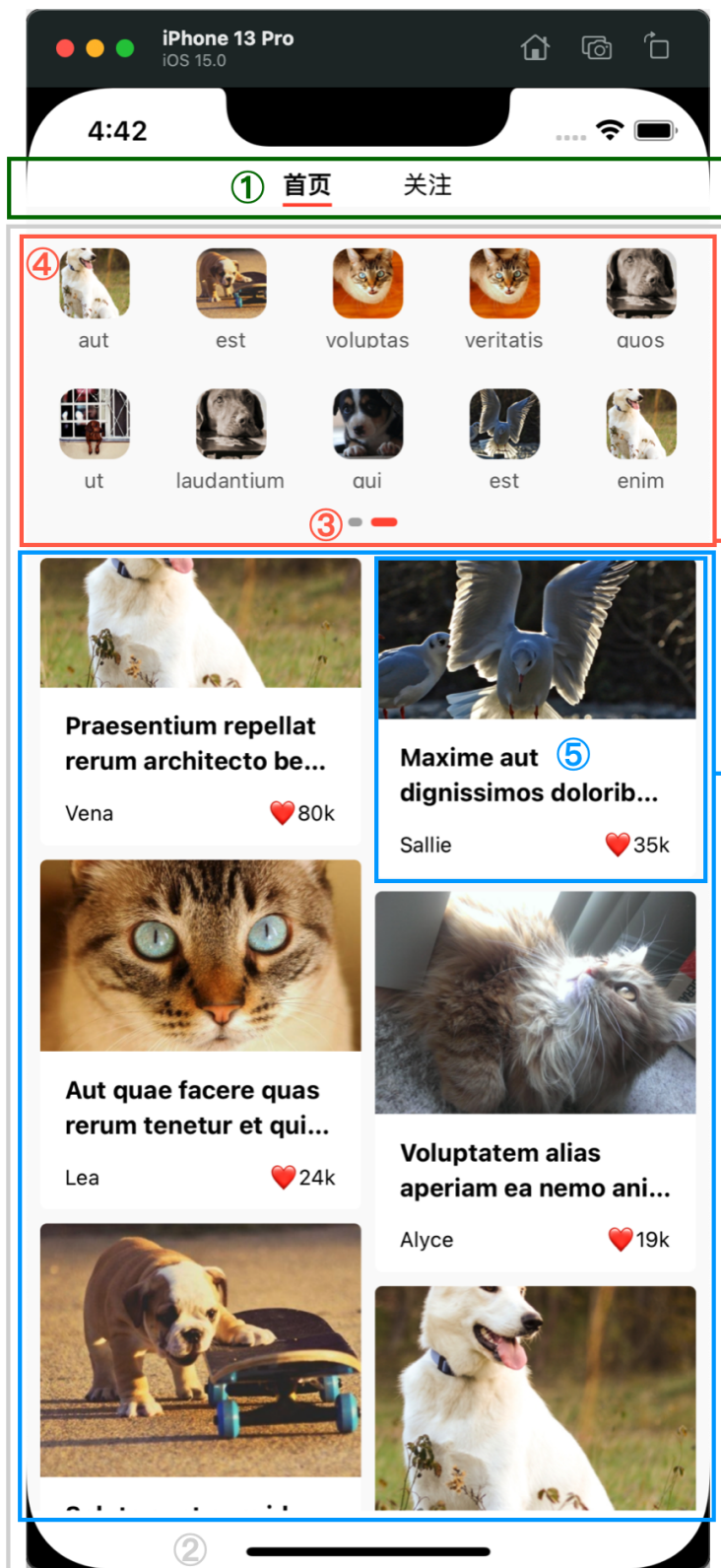
这两个通用组件是网格布局组件 Grid，和瀑布流版的 RecyclerView，我把它们放到了 components 目录下：

```
├── components
│   ├── Grid
│   └── RecyclerView
```

有了上面两个通用组件后，我就只需要专注于页面的开发即可。Grid 组件对应金刚区图标的 2*5 的网格布局。不过，瀑布流版的 RecyclerView，也不是拿来就能用的，比如金刚位和瀑布流的混合列表，我是上一讲的基础上改了改代码才实现的。

要开发页面，就要先把它拆成组件。在[02讲](#)中，我提到过拆组件原则是单一职责原则，一个组件只做一件事，我还在[04讲](#)说过，组件的状态根据就近原则进行放置，我们应该先考虑放在该组件上，再去考虑父组件。

根据单一职责原则和就近原则，NFT 首页的设计稿我是这么拆的：



- TopBar (顶栏)
= View + Text
①页面切换的状态

- List (无限列表)
= RecyclerView
②加载状态

- Icons (金刚位)
= Grid + ScrollView
③滑动切换
④图标内容

- WaterFallCard (瀑布流卡片)
= View + Text
⑤卡片内容

我们先来具体分析一下 TopBar 。

顶栏就是 TopBar 组件，咱们基础篇先不考虑动画、手势，因此 TopBar 组件使用最简单 View 和 Text 就能实现。其中有个麻烦的地方，页面切换的状态应该放在哪里？

我是这么思考的，根据就近原则我先把页面切换的状态放到了 TopBar 组件上面：

- App(应用)
 - Home(首页)
 - TopBar(顶栏) <- 页面切换的状态
 - Follow(关注页)

但实际上，你可以看到页面切换的状态其实并不属于 TopBar 组件，也不属于它的 Home 父组件。它是 App 组件用来控制 Home 组件和 Follow 组件切换的状态，因此它应该属于 App 组件。而这次我负责开发的是首页，所以我先把暂时放在了 TopBar 组件上，后续我和负责 App 开发同学联调的时候，再把状态抽到 App 组件的全局状态上。

接着是List的实现。无限列表 List，底层直接使用瀑布流版的 RecyclerView 实现就可以了。

而且，无限列表的加载状态，我们也在[04讲](#)中提到过，所有的 isLoading、isError、isSuccess 都可以合并成一个状态：

```
enum RequestStatus {  
    IDLE = 'IDLE',  
    PENDING = 'PENDING',  
    SUCCESS = 'SUCCESS',  
    ERROR = 'ERROR',  
}
```

除了 RequestStatus 这种枚举类型的方案外，当然还有更简单的方案，在单个组件的分析维度，我会再和你介绍。

再接着是 Icons。

金刚位 Icons，我用我自己开发的网格组件 Grid 和滚动组件 ScrollView 就能实现。ScrollView 组件我们也在[08讲](#)中介绍过，我打算用它的横向滚动、分页能力和滚动结束事件，来实现金刚位的支持左右滑动切换、双屏切换的功能。

接下来我们还会用到这些 [ScrollView](#) 属性：

- horizontal
- pagingEnabled
- onMomentumScrollEnd
- showsHorizontalScrollIndicator

这里我们再简单解释下这几个属性：

horizontal 默认为 false，是竖向滚动的，将其设置为 true 时，即可开启横向滚动；

pagingEnabled 默认为 false，是滚动交互是平滑的，将其设置为 true 时，每滚动一次就翻一页。horizontal 和 pagingEnabled 同时开启的效果类似轮播图；

onMomentumScrollEnd 是滚动结束事件，当滚动停下来时会触发一次。金刚位的滑动翻页时，有一个长一点的小红条和一个短一点的小灰条，用来表示当前显示的那一屏。使用 onMomentumScrollEnd 就可以控制滑动切换状态了；

showsHorizontalScrollIndicator 默认为 true，代表默认显示横向滚动条，金刚位的轮播图效果不需要滚动条，因此我准备把它关了。

金刚位有两个状态，滑动状态和图标内容状态。滑动状态我刚刚也提到过，只有 Icons 本身在用，因此我们直接放到 Icons 组件上即可。而图标内容状态，因为 RecyclerView 不像 ScrollView，RecyclerView 的 dataProvider 是统一维护的，所以我打算把图标内容状态移到 Icons 的父组件 List 上。

最后是 WaterFallCard 。

前面我们提到过 NFT 首页是由金刚位和瀑布流组成的混合列表，我们不能用 RecyclerView 嵌套 RecyclerView 来实现混合列表。这里我敲一个重点，RecyclerView 是继承自 ScrollView 的，同一个方向也就是垂直方向或水平方向，我们尽量只使用一个 ScrollView/RecyclerView 组件来进行响应。

那为什么我选择只用一个 ScrollView/RecyclerView 呢？

你这样想，同方向的双 ScrollView 有两种响应方式，同时响应或只响应一个。只响应一个的时候，双 ScrollView 和单 ScrollView 是一样的。第二种情况是双 ScrollView 同时手势响应，你可以想象一下，在你用鼠标往上滚动页面时，还有一个调皮的小朋友用你的键盘控制页面往上滚动。第二个 ScrollView 组件，就像那个调皮的小朋友，你在动的同时他也在响应你的手势，结果滚动速度就变成双份的了，格外奇怪。

而且，同方向的双 ScrollView 并不能实现金刚位和瀑布流的混合列表，因此我选择了采用改 RecyclerView 的源码，让它同时支持单列布局和双列瀑布流布局，这就要一些技术攻坚了。怎么改第三方库的 JavaScript 源码，我们也在[11讲](#)中学习过。

正是因为，瀑布流实际不是列表，它只是无限列表 List 组件中的卡片，因此我将其命名为 WaterFallCard。实现 WaterFallCard 组件，需要用到 View、Text、Image、Pressable 组件，此外还要用到 [03讲 Style 样式](#)的知识，这一部分实现起来会比较简单。

单个组件

当我把 NFT 页面拆成 2 + 4 个组件后，我的实现思路就清晰很多了。两个通用组件，不需要什么改动，工作量很小，4 个业务组件只有列表 List 组件状态管理比较麻烦，这时候我就把重点放到了 List 组件上。如果你想了解 4 个业务组件的具体实现，也可以看下我放在 [GitHub](#) 上的代码。

如果做过无限列表，你就知道，处理里面的逻辑还挺麻烦的，需要处理首次请求成功、首次请求失败、更多数据加载成功/失败、后端没有数据等等情况，如果考虑性能优化的话，还要做预加载、要管理数据缓存的逻辑，要写很多代码。

这时候，我想起了以前在技术群里有朋友推荐过的 React Query，说是处理请求状态非常简单，以前就简单看过一下 API，它的 Demo 代码如下：

```
import { QueryClient, QueryClientProvider, useQuery } from 'react-query'

const queryClient = new QueryClient()

export default function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <Example />
    </QueryClientProvider>
  )
}

function Example() {
  const { isLoading, error, data } = useQuery('repoData', () =>
    fetch('https://api.github.com/repos/tannerlinsley/react-query').then(res =>
      res.json()
    )
  )
}
```

```

    )
  )

  if (isLoading) return 'Loading...'

  if (error) return 'An error has occurred: ' + error.message

  return (
    <div>
      <h1>{data.name}</h1>
      <p>{data.description}</p>
      <strong>👁️ {data.subscribers_count}</strong>{' '}
      <strong>🌟 {data.stargazers_count}</strong>{' '}
      <strong>🔗 {data.forks_count}</strong>
    </div>
  )
}

```

代码行数不多，虽然是 Web 代码，但我相信你也能很容易看懂。QueryClient 和 QueryClientProvider 让我感觉和 Redux 状态管理库似曾相识，应该是管理全局状态用的。但 useQuery 这段代码非常简洁，把网络请求状态统一封装了，加载中是 isLoading，加载报错是 error，请求数据是 data，能够省去很多模板代码。看完后，当时我心里就想，这代码抽象得真好。

这次写文章时，我又想起了 React Query，于是又去仔细研究了一下，第一个惊喜是它的竞品 SWR 的代码，竟然比 React Query 还要简单，你可以看下 SWR 提供的 demo：

```

import useSWR from 'swr'

function Profile () {
  const { data, error } = useSWR('/api/user/123', fetcher)

  if (error) return <div>failed to load</div>
  if (!data) return <div>loading...</div>

  // render data
  return <div>hello {data.name}!</div>
}

```

默认没有全局配置的 Client 和 Provider 的配置，只用了一行 useSWR，就把请求逻辑处理完成了，并且只提供了两个状态数据 data 和 error，isLoading 和 !data 是等价的，于是又把 isLoading 状态省掉了。作者真是高手，后来我一看是 Next.js 团队开发的，心里好感又增加了一些。

第二个惊喜是 React Query 和 SWR 竟然针对无限列表这种场景，做了通用的封装，这又能进一步把 List 组件的状态管理逻辑降低。以 React Query 为例，示例代码如下：

```
const {
  fetchNextPage,
  fetchPreviousPage,
  hasNextPage,
  hasPreviousPage,
  isFetchingNextPage,
  isFetchingPreviousPage,
  ...result
} = useInfiniteQuery(queryKey, ({ pageParam = 1 }) => fetchPage(pageParam), {
  ...options,
  getNextPageParam: (lastPage, allPages) => lastPage.nextCursor,
  getPreviousPageParam: (firstPage, allPages) => firstPage.prevCursor,
})
```

不过，我这里要先和你坦个白，[查文档](#)看这段代码的时候，我也没有看得太懂，状态太多了，还有个配置项，我其实是在真正用的时候，才用明白的。

我从文档中大概能够理解，useInfiniteQuery 是专门用来控制无限列表请求的状态的，里面状态很多，控制状态的函数也很多。fetchNextPage 从名字看是获取下一页，hasNextPage 是是否有下一页的意思，isFetchingNextPage 是是否正在请求下一页的意思，但 getNextPageParam 和 getPreviousPageParam 完全没有看懂，特别是 nextCursor 属性是什么、干什么用的，也没有搞明白。

同样，在 SWR 的中文文档中，处理无限数据列表的 useSWRInfinite 函数我也看了一遍，汉字都认识，代码单词都认识，就是不理解。

或许你曾经也遇到过和我类似的问题，看别人的文章、文档懂了个大概，但心里还是困惑很多。这个时候，抽象的文字要和能跑得起来的代码结合起来读，读完

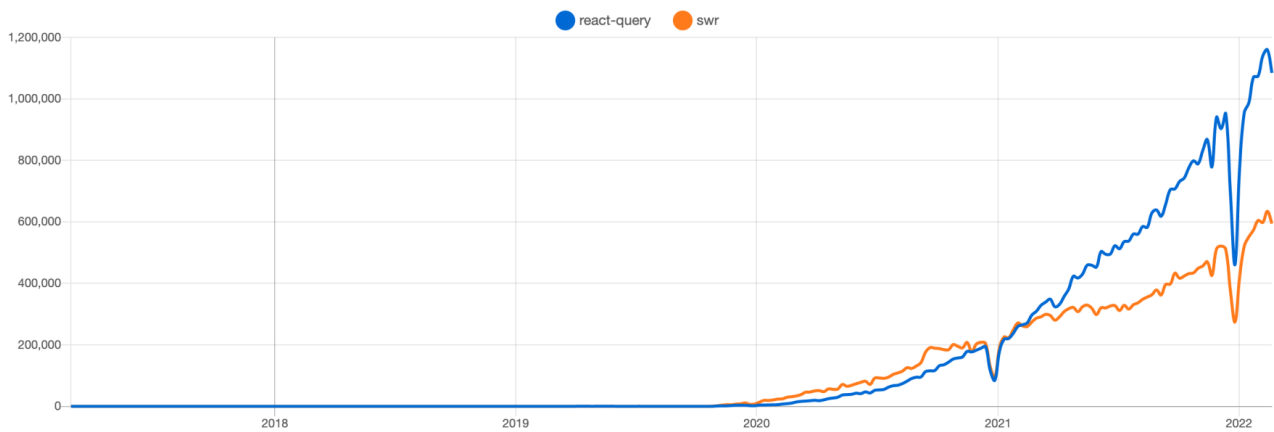
useInfiniteQuery/useSWRInfinite [示例代码](#)后，我自己写了一个 React Native 的版本之后，我才真正算会用了。

但在当时，我要解决的核心问题不是怎么把 useInfiniteQuery/useSWRInfinite 学会，我的计划是在 1 天的周末时间把简易电商首页搭建出来，所以了解完 React Query 和 SWR 的核心功能后，我得快速决策 List 组件怎么技术选型。

在当时我是这么决策的，React Query 和 SWR 都支持 React Native，在 React Query 官方出的和 SWR 的[对比分析文章](#)中，React Query 似乎更强大一些，但多出的功能我并不会用到，在功能方面二者打了个平手。

然后我又分析了一下文档，SWR 官方支持中文，React Query 只能找到英文文档，但 React Query 功能丰富度比 SWR 更强大一些，我又对比了一下 [npm trends](#)，发现大家更喜欢 React Query 一些，从趋势图中你可以看出 React Query 在 2021 年的下载量反超了 SWR。

Downloads in past 5 Years



Stats

		Stars	Issues	Version	Updated	Created	Size
react-query	npm github docs	25,664	30	3.34.16	8 days ago	8 years ago	minzipped size 12.9 KB
swr	npm github docs	21,300	67	1.2.2	10 days ago	4 years ago	minzipped size 4.2 KB

中文英文文档我没有什偏好，但文档详细度、Demo 数量和 React Query 的下载量，实打实是 React Query 更优秀，因此我快速做出了决定，选 React Query。

总结

好了到现在为止，从项目维度、页面维度和单个组件维度，我把一些关键点都分析完了，排除难点有了思路后，接下来就是写代码了，都是体力活了。

代码如何写，细节是什么，我就不和你详细介绍了，因为写这篇文章的目的其实是为了让你通过搭建一个电商首页，检验自己学到了多少知识。老师教了多少不重要，你学到了多少才是最重要的。

这篇文章也是我对自己一次自我剖析，在这篇文章中，你也能看出我的整体思路，是把“如何搭建一个电商首页？”这个大问题，分解为“如何搭建项目”，又进而分解为“如何拆分页面”，最后分解为“如何攻克单个复杂的组件实现”，在不同思考层面宏观、中观、微观，利用自己的经验和从优秀开源库中学到的实践，来解决不同层面的问题。

但老师也有他的技术盲区，有些业内已经解决的问题，我以前都没有意识到这是个问题，还在用着我以前复杂的解决方案，比如 React Query 我真是今年才听过，这次写代码的时候才用上的。

但因为以前这种电商首页写过很多个，无限列表组件也封装过，看到 React Query 就明白它大致解决的问题是什么，能解决我以前的哪些问题，也就可以现学现卖了。

这也让我更加深刻地意识到，知识本身不重要，要把知识内化成自己的能力才重要。只有以前真正思考过、实践过才能在遇到类似问题时，遇到更好解决方案时，快速学习、决策和解决。

作业

这一讲的作业，就是搭建一个卖 JPG 的电商首页。

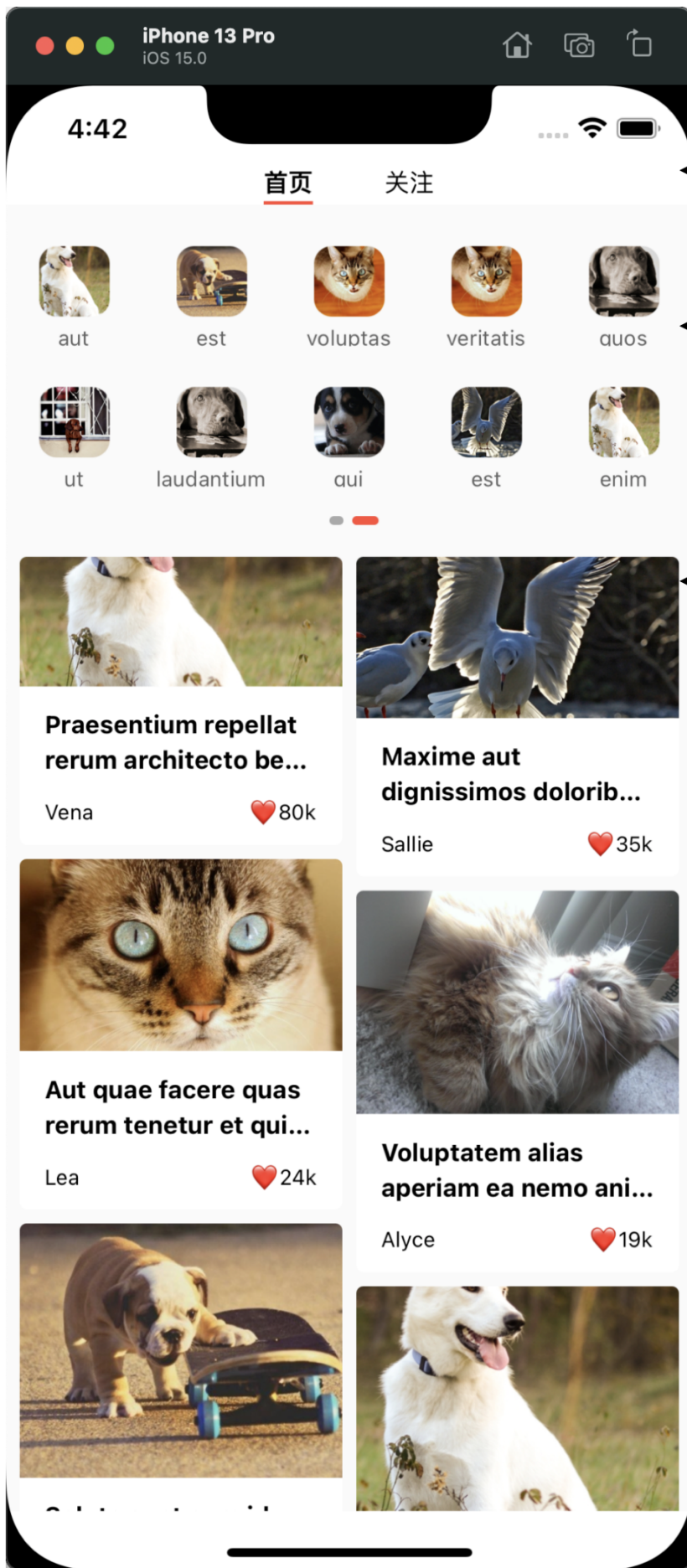
这次作业是对技术篇所学知识的一种巩固，技术篇中没有用到的技术，比如动画、页面跳转，你可以先忽略，后面我们还会继续完善这个电商 App。有些绕不开的技术，如果你以前不会，我也没有教过，你可以试着自己查查资料，一边看文档和代码案例，一边自己动手试试，相信你也能很快掌握。

这次没有搭建专门的后端服务，所有的数据、图片用的都是 mockapi.io 的假数据，这些假数据接口如下：

```
const animalsUrl = 'https://61c48e65f1af4a0017d9966d.mockapi.io/animals';  
const catsUrl = 'https://61c48e65f1af4a0017d9966d.mockapi.io/cats';  
const iconsUrl = 'https://61c48e65f1af4a0017d9966d.mockapi.io/icons';
```

第一个 animalsUrl 是动物的图片数据，第二个 catsUrl 是猫猫的图片数据，你可以任选其一作为瀑布流的展示数据，第三个 iconsUrl 是金刚位的图标数据。

请你使用上面三个接口，实现如下 UI 设计稿：



顶栏

用于切换首页和关注页

金刚位

由 $2 * 10$ 个图标组成
支持左右滑动切换

瀑布流

由高度不定的卡片组成
卡片内容包括：

1. JPG 本身
2. JPG 的座右铭
3. 作者和点赞人数

如果你在实现过程中遇到了问题，花了大量时间也没有想明白的时候，你可以参考一下我放在 [GitHub](#) 上的代码，希望它能对你有所帮助。如果还有其他问题，也欢迎你给我留言。我是

蒋宏伟，咱们下一讲见。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 11 | 组件实战：如何实现瀑布流？

下一篇 直播加餐 | 七年，我的跨端实践和探索

精选留言 5